

LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. **AI APPLICATIONS
IN PRACTICE**

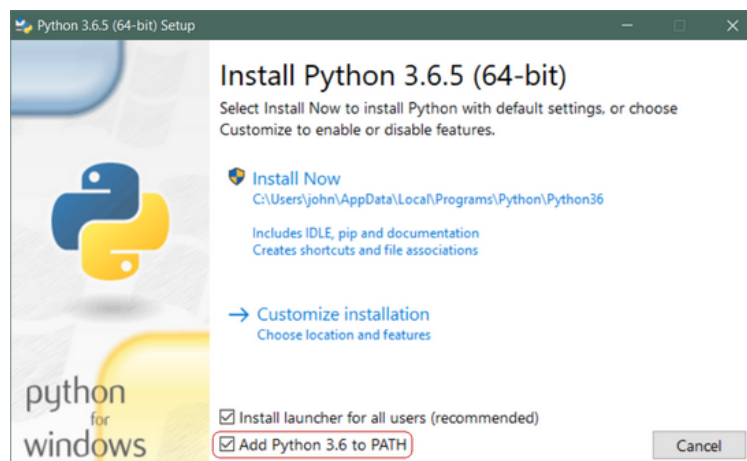
1 - Python Basics

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

Getting started - Installation

Windows

- To install Python 3 go to <https://www.python.org/downloads/> and download the executable for the latest version
- Run the installer, just be sure to add Python to the system environment variables



Getting started - Installation

Linux

- Python is probably already installed!
- Check it using a terminal by executing:

python --version
(also ***python2 --version*** and ***python3 --version***)

If it is not installed you can do so like this (Ubuntu):

sudo apt-get update
sudo apt install software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt-get update
sudo apt install python3.12

3

Getting started - Installation

Mac

- Python is probably already installed!
- Otherwise you can install it using brew:

brew install python3

You will need Homebrew installed, you can do that with:

/usr/bin/ruby -e "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"

4

Getting started - IDE

Several **Integrated Development Enviroments (IDE)** exist:

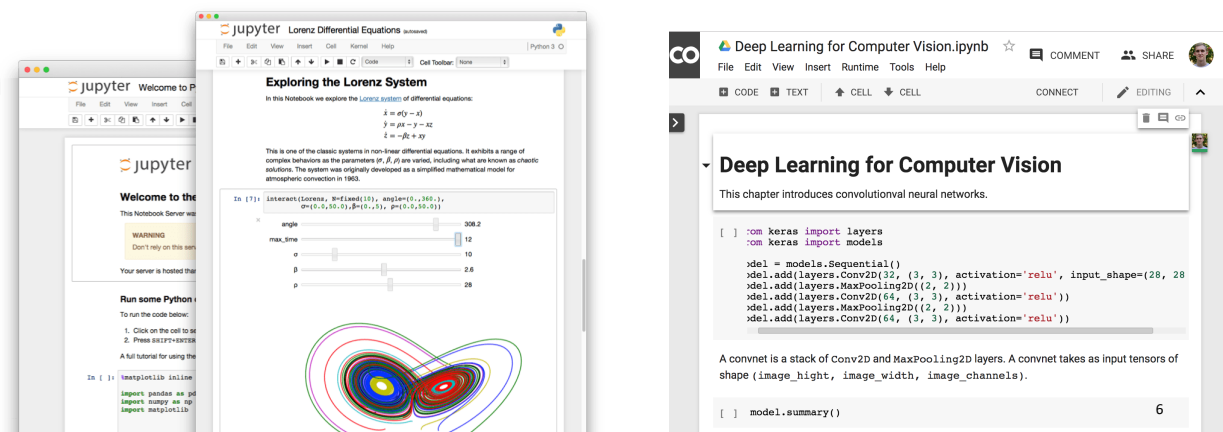
- Jupyter Notebooks/Google Colab
- Visual Studio Code
- Thonny
- IDLE

5

Getting started - IDE

Jupyter Notebooks / Google Colab

Notebooks allow us to run code in a web page using interactive cells containing code, text, images, plots, equations, ecc.



Getting started - IDE

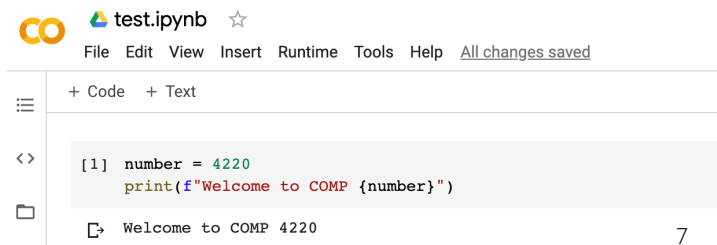
Google Colab

Google Colaboratory, or “Colab” for short, allows you to write and execute Python in your browser

<https://colab.research.google.com/>

- Zero configuration required
- Free access to GPUs and other computing resources
- Use terminal commands on Google Colab

<https://drive.google.com>
(for the first time, select
“Connect more apps”)

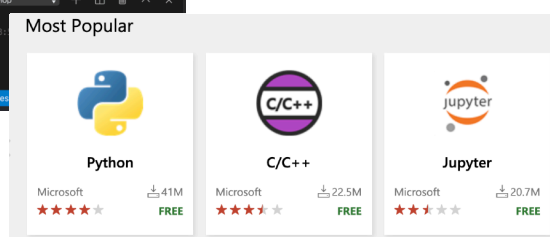
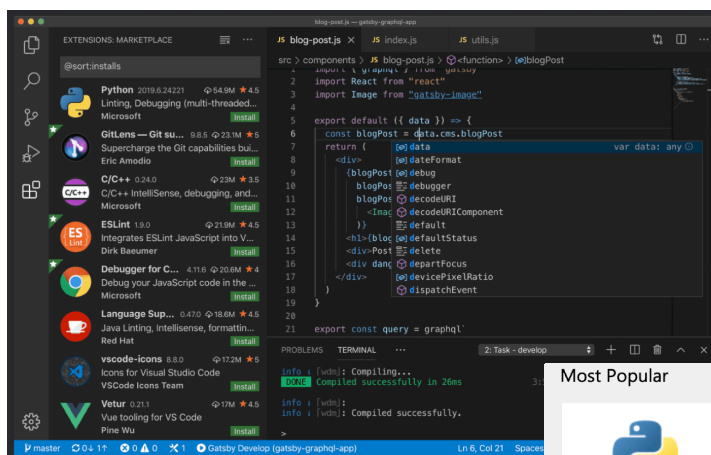


Getting started - IDE

Visual Studio Code

<https://code.visualstudio.com/>

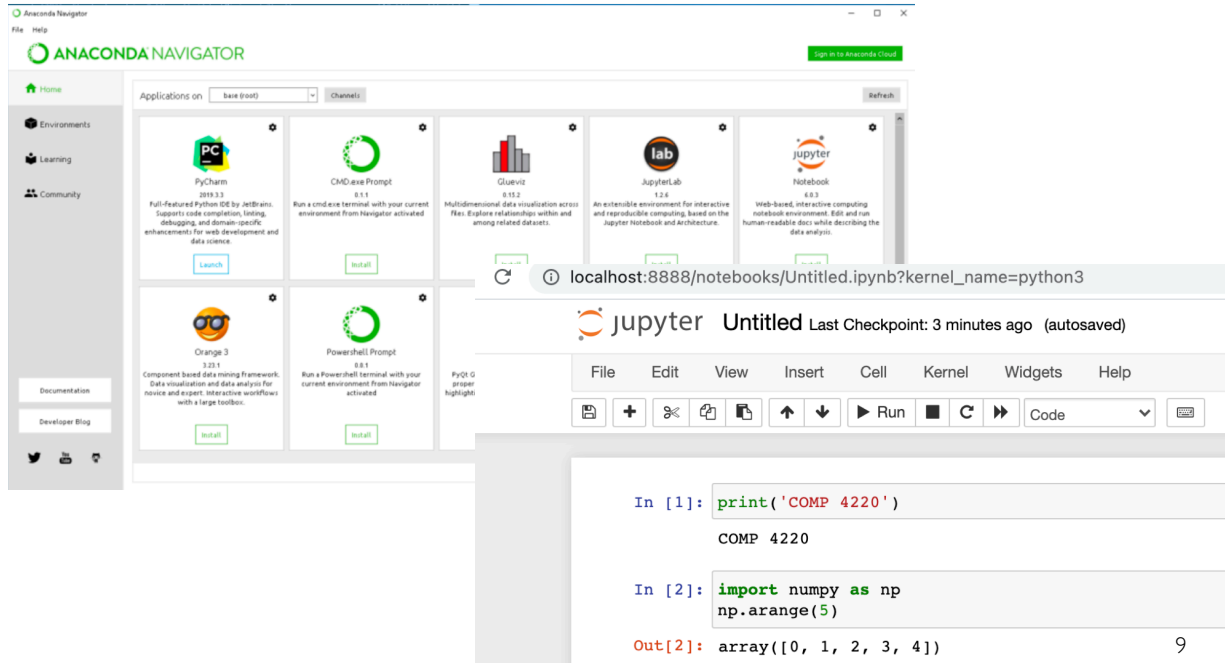
- Professional flexible IDE that allows to work on local and remote projects.



Getting started - IDE

Anaconda

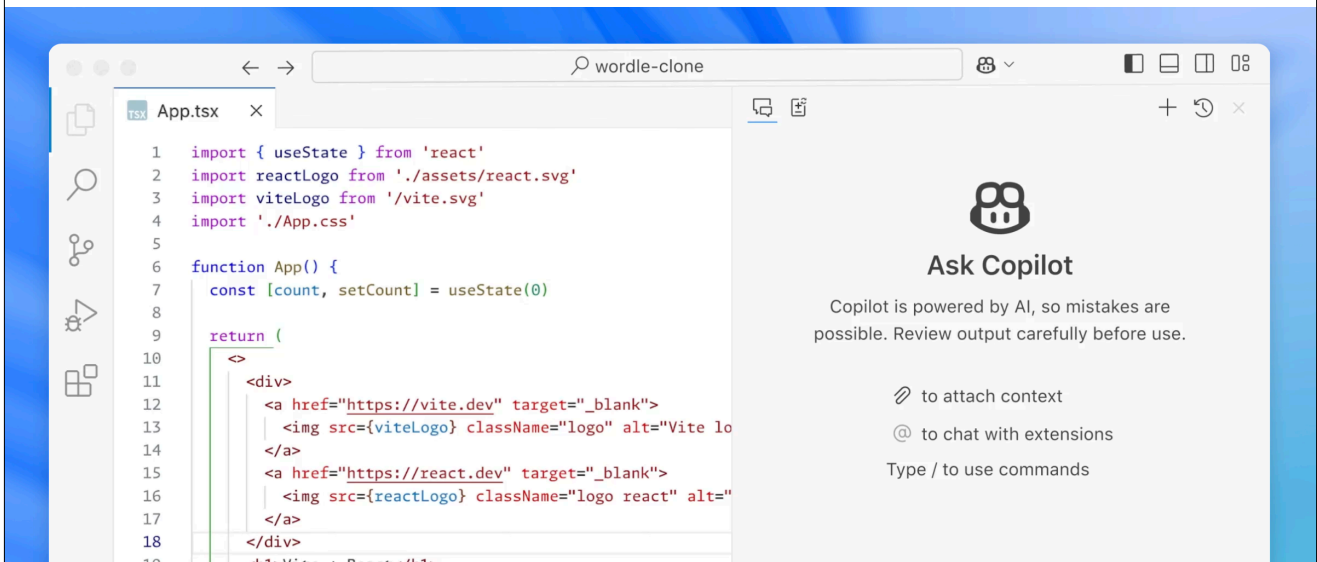
www.anaconda.com



ENVIRONMENT SETUP

Vscode

Download and install Visual Studio Code from
<https://code.visualstudio.com/>

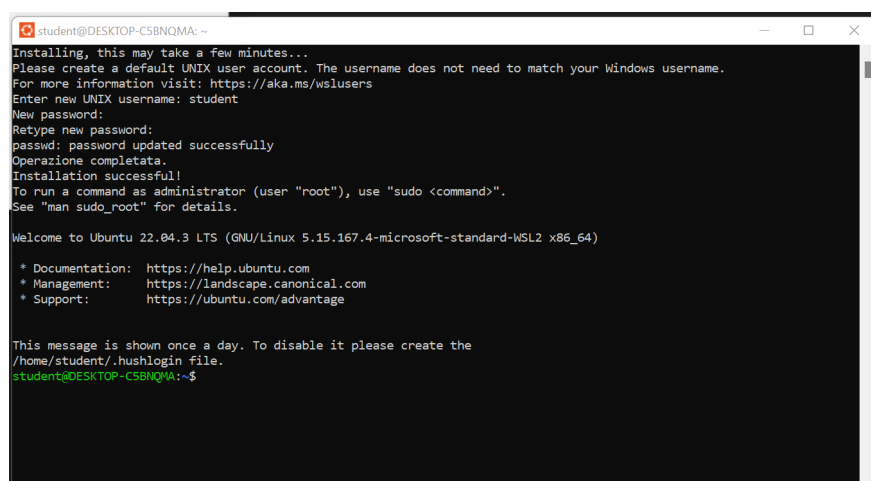


WSL



On the lab's computers the preferred way to test Python and the other software will be using Windows Subsystem for Linux.

WSL provides access to a Linux instance, where you can be root and responsible for making it working.



WSL

You can install Ubuntu 24.04 from the Microsoft Store.
For the course, we provide a fresh Ubuntu 24.04 instance that has minimally refined. You can import it on your or lab's computer:

```
mkdir C:\wslDistroStorage\Ubuntu
```

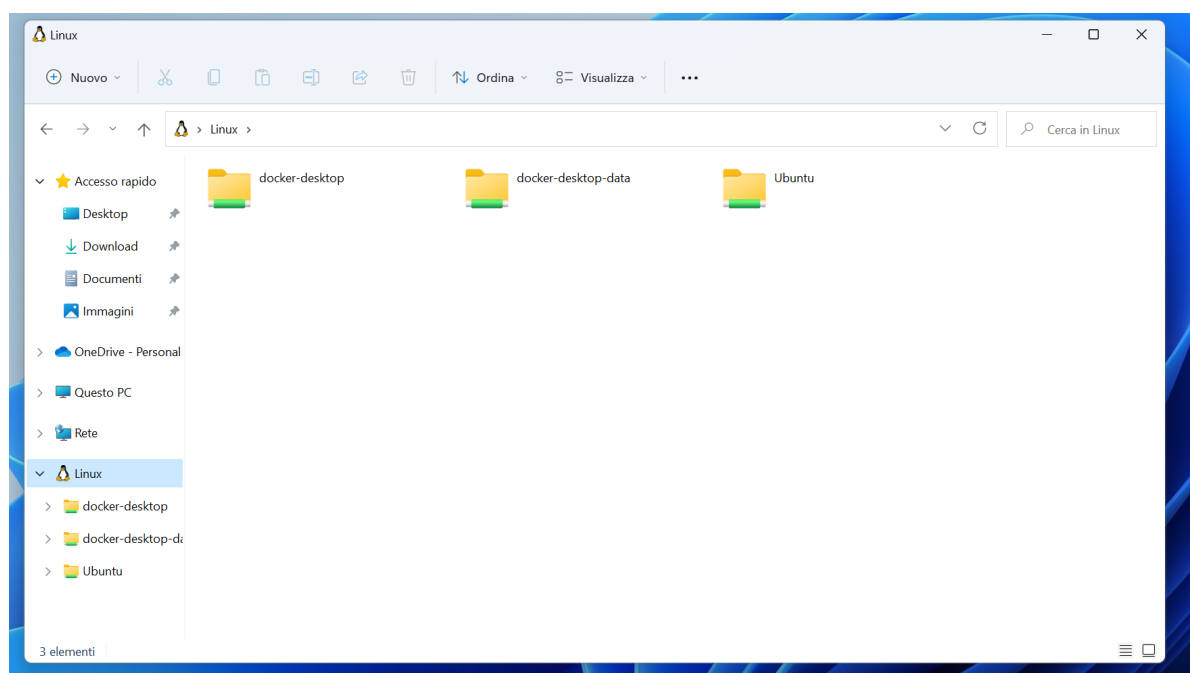
```
wsl --import Ubuntu-LabInformaticaApplicata C:\wslDistroStorage\Ubuntu .\ubuntu-lab_informatica_applicata.tar
```

```
wsl
```

13

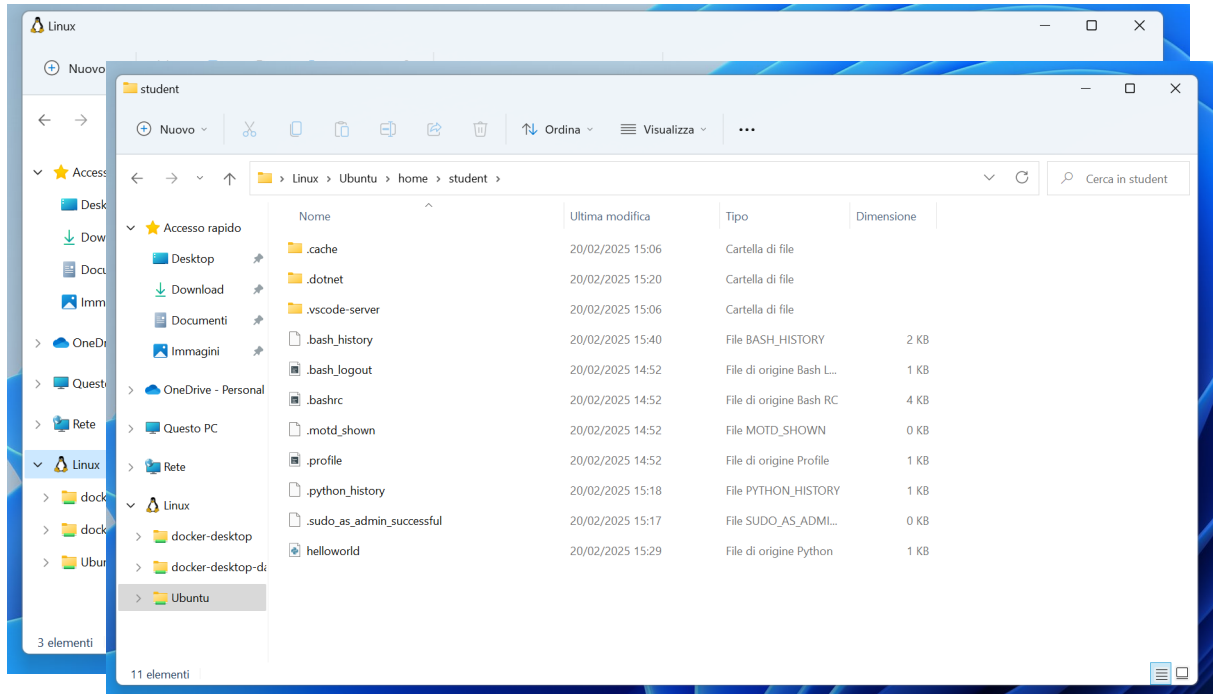
Accessing WSL Filesystem

WSL is a proper linux virtual machine running in background.
You can access its file system directly from Windows Explorer.



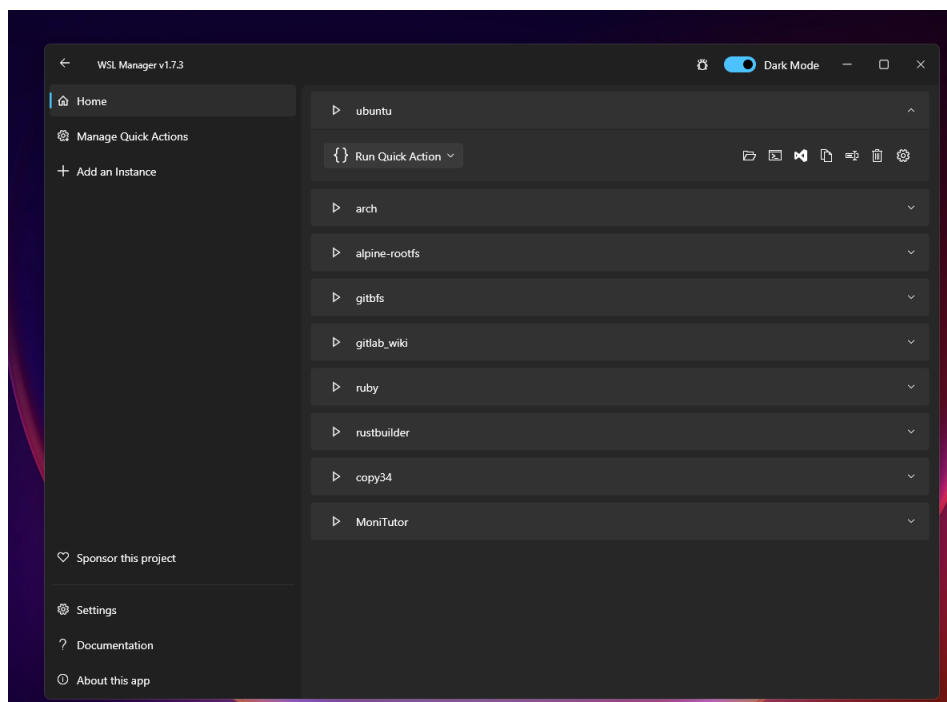
Accessing WSL Filesystem

WSL is a proper linux virtual machine running in background.
You can access its file system directly from Windows Explorer.



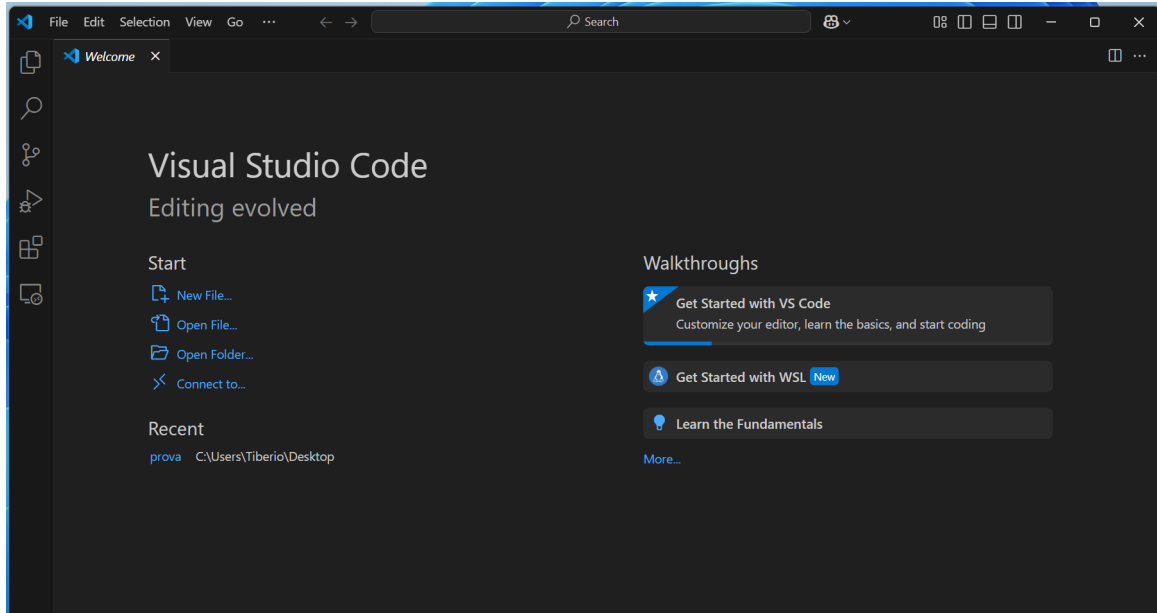
WSL distro manager

WSL supports multiple distributions, you can easily manage them with this tool: <https://github.com/bostrot/wsl2-distro-manager?tab=readme-ov-file>

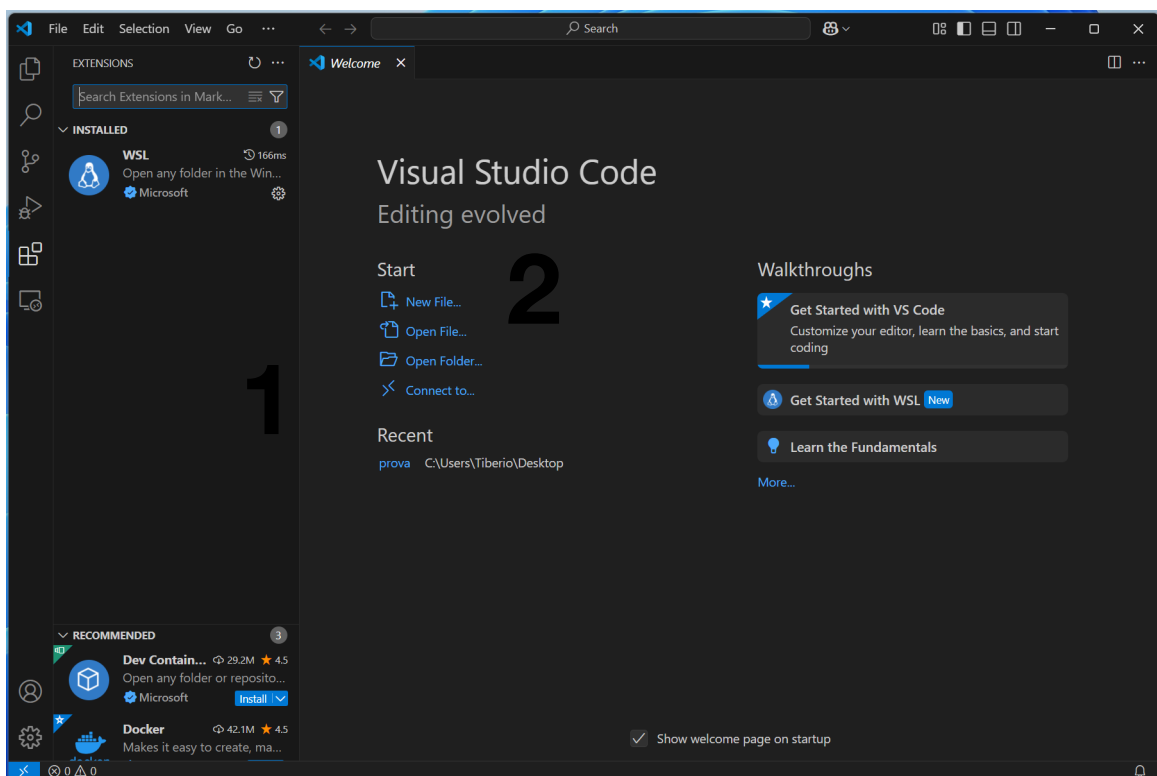


Configure VSCode

Once WSL is ready, start VSCode, configure its extensions and test the environment.



Configure VSCode

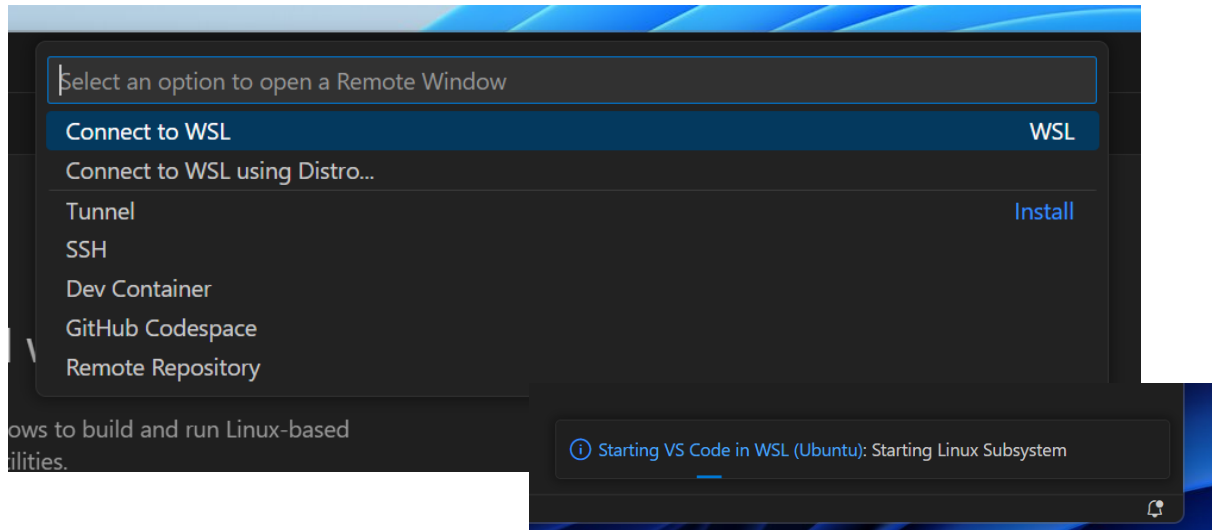
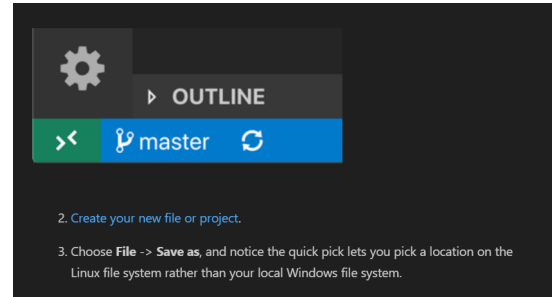


Configure VSCode

Ctrl+Alt+O

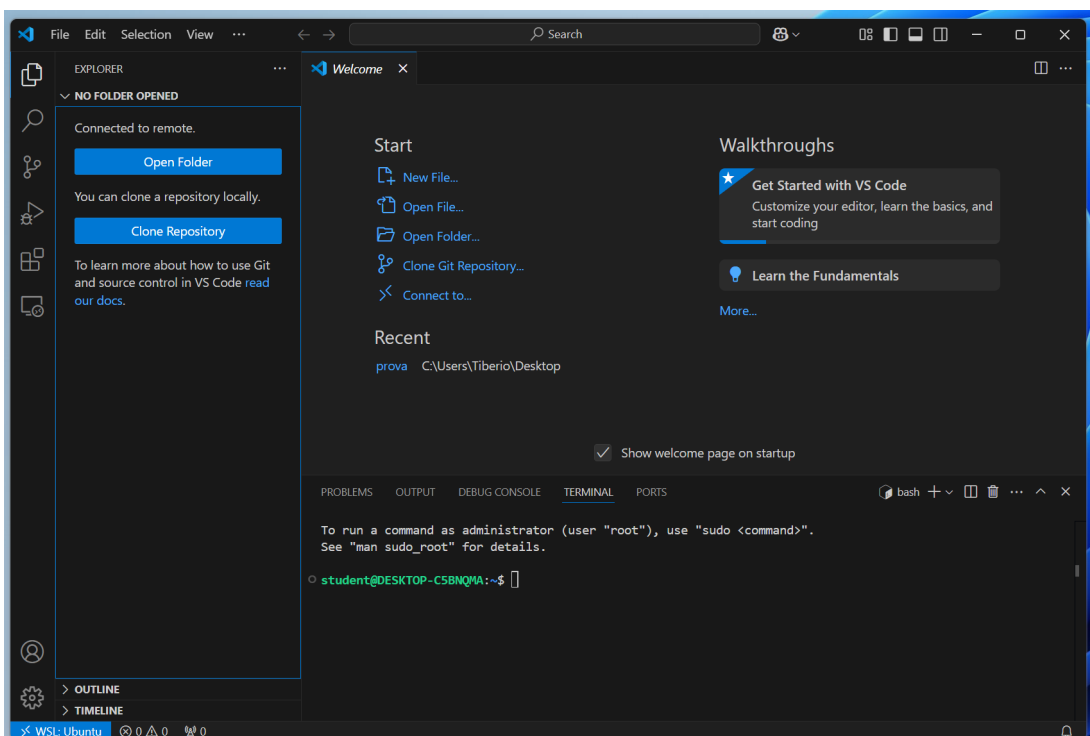
Cmd+Alt+O

WSL Distro: Ubuntu



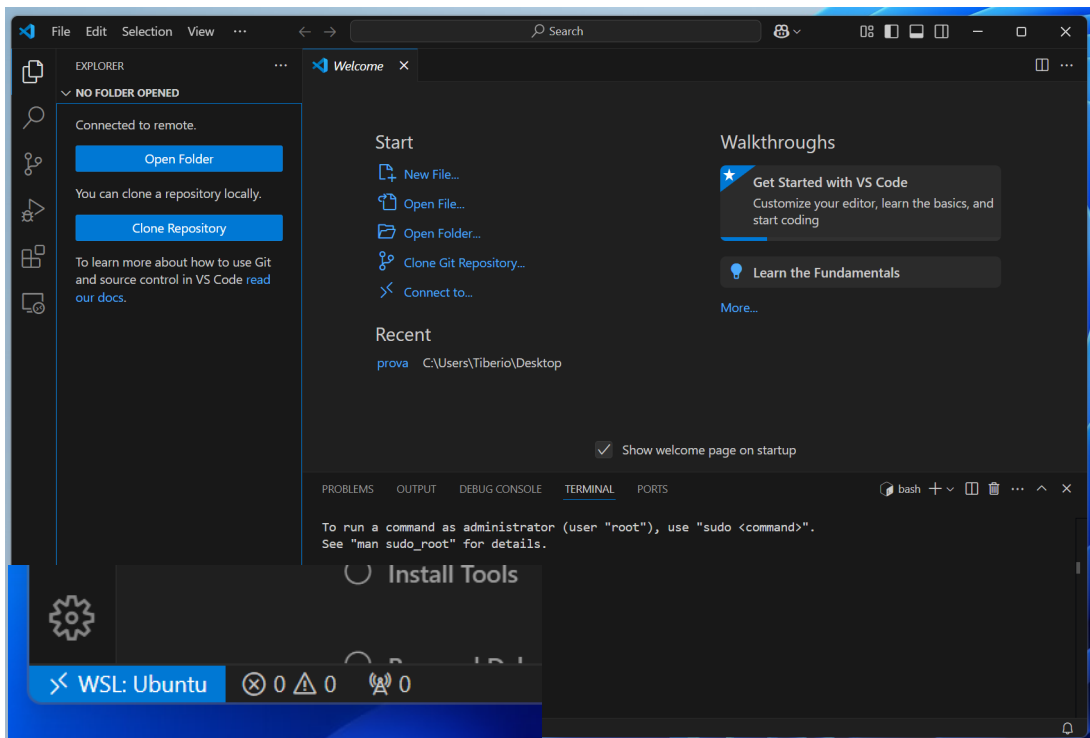
Configure VSCode

If everything went fine, after a minute you will have connected to the instance.



Configure VSCode

If everything went fine, after a minute you will have connected to the instance.

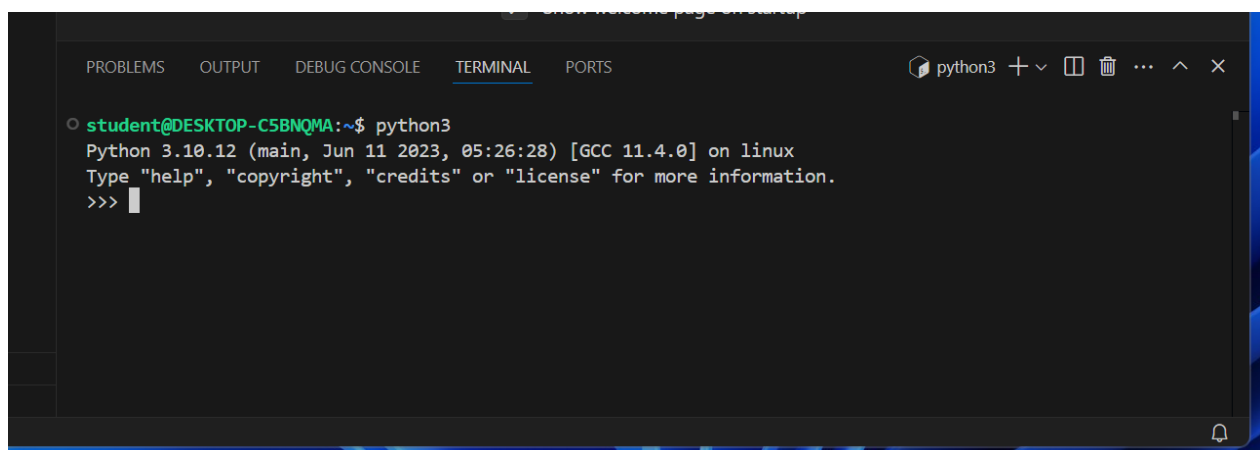


21

Try to start Python

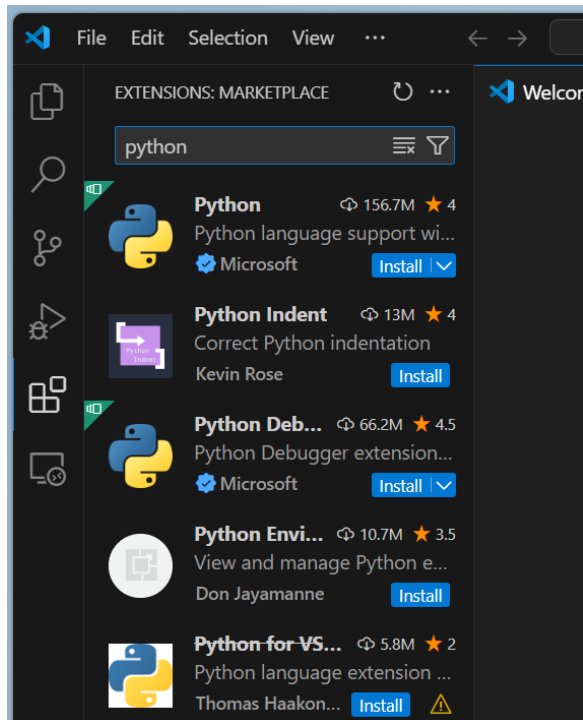
Python has a shell that interprets code.

You may start it by launching `python3`



22

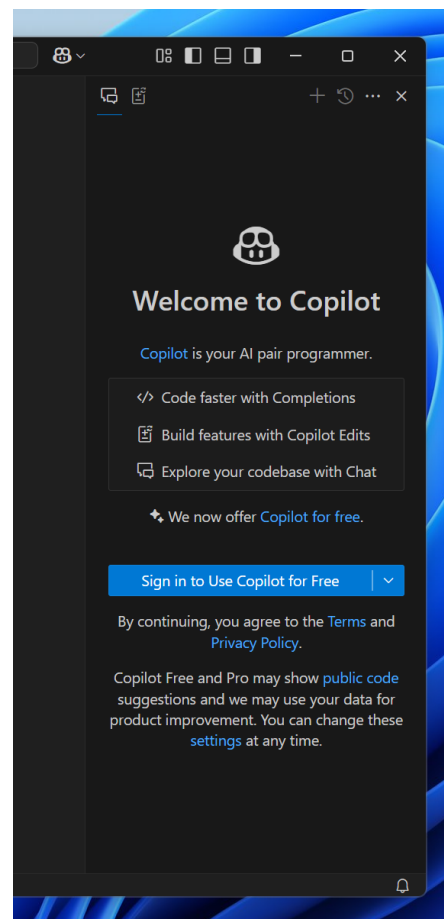
Install Python Extension



23

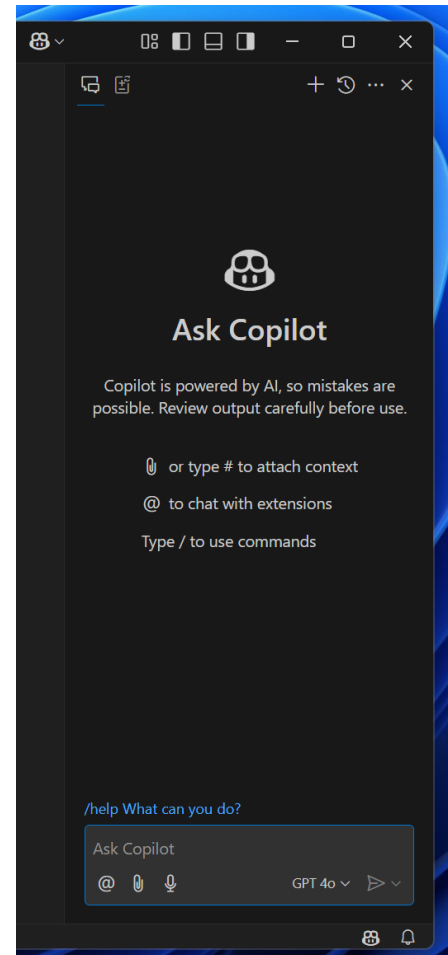
Enable Copilot

VSCode provides integration with GitHub Copilot, an AI system that provides completions of code while writing, and can write code for you.



Enable Copilot

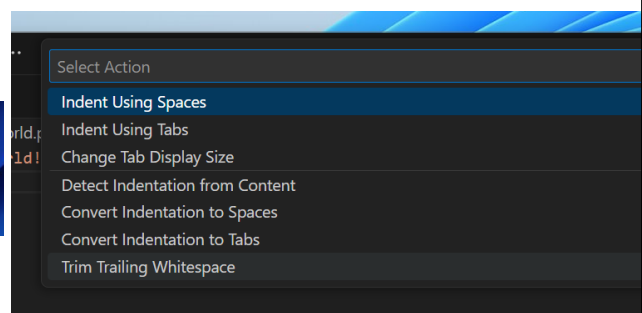
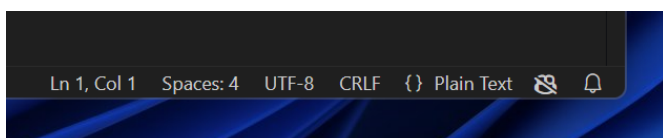
After installation you can use Copilot functions from the same menu



Spaces or tab?

In the famous war between spaces and tabs, the Python style guide (PEP 8) recommends using spaces over tabs - **four space characters** per indentation level.

However, the same PEP 8 recommends that, if you're working with a code that's already using tabs, you can continue using them to keep the indentation consistent.

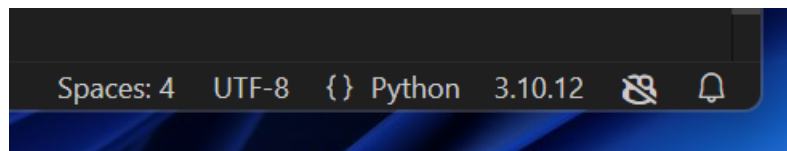


Python distributions and versions

You may have several versions of Python installed in a system, along with its specific packages.

That is very common since you can have packages requiring specific versions, or you may deploy your own app in production with tested versions.

Be aware of which Python is currently used in VSCode!

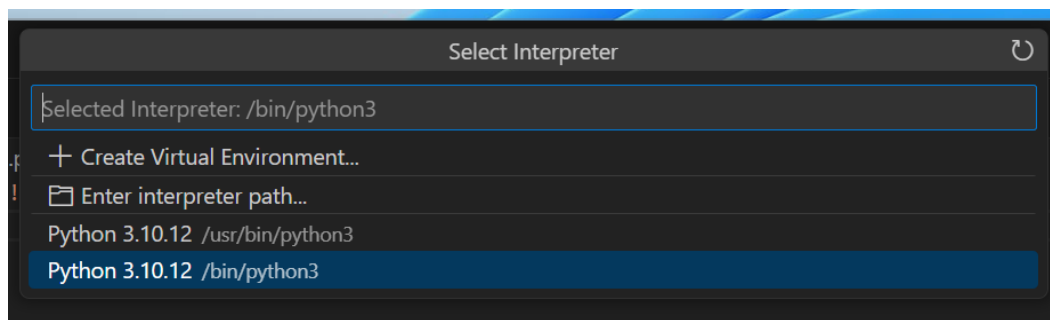


27

Python distributions and versions

VSCode shows the interpreters that are available for every users on the system.

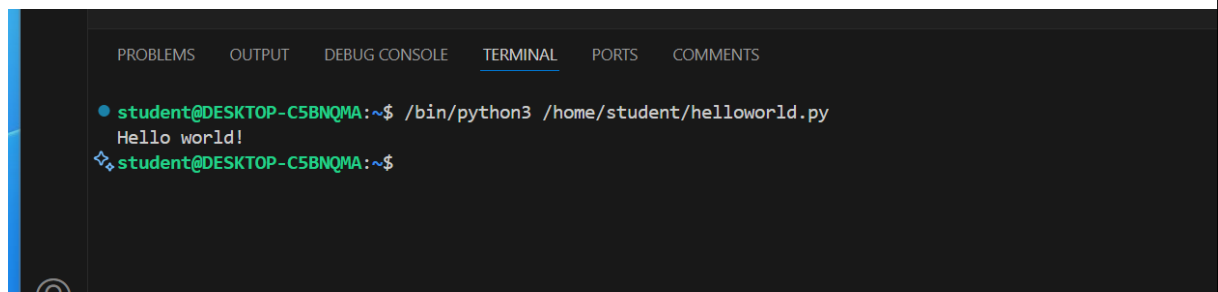
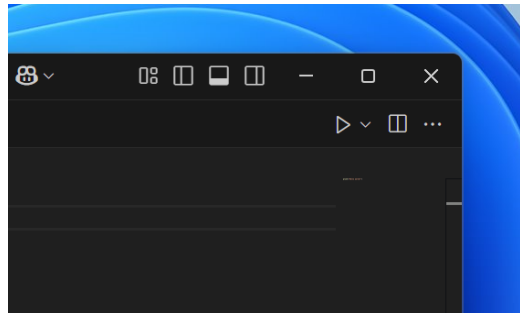
There is also the possibility to create *Virtual Environments*, that are isolated environments with specific Python versions and installed packages.



28

Running in VSCode

Run the opened file by clicking the Play button

A screenshot of the VS Code interface with the 'TERMINAL' tab active. The terminal shows the command `/bin/python3 /home/student/helloworld.py` being executed, which outputs `Hello world!`. The prompt is `student@DESKTOP-C5BNQMA:~$`. The terminal has a dark background with green and blue text.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS  
● student@DESKTOP-C5BNQMA:~$ /bin/python3 /home/student/helloworld.py  
Hello world!  
❖ student@DESKTOP-C5BNQMA:~$
```

INTRODUCTION TO PYTHON

What is Python?

- High-level programming language, interpreted and dynamic
- Simple and readable, with a clear and minimal syntax

Why use Python?

- It's free!
- It uses multiple paradigms (procedural, objects, functional)
- Portable (Linux, Windows, macOS, ...)
- Easy to use
- Plenty of libraries
- Has good performance (*even almost native if well used)
- Automatically handles memory management
- Interoperable with other languages
- Simple and readable, with a clear and minimal syntax

Who uses Python?

- NASA is employing python for control systems
- Yahoo! Developed several internet services in python
- Google, Youtube, RedHat, all abundantly use python

Most used languages

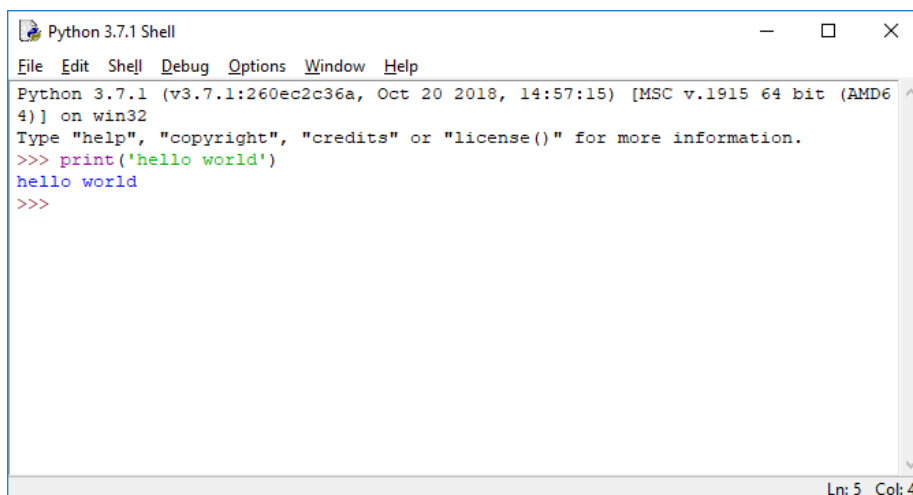
Philosophy of Python (The Zen of Python)

- PEP (Python Enhancement Proposal):
<http://www.python.org/dev/peps/pep-0020/>
- 1. Beautiful is better than ugly.
- 2. Explicit is better than implicit.
- 3. Simple is better than complex.
- 4. Complex is better than complicated.
- 5. Flat is better than nested.
- 6. Sparse is better than dense.
- 7. Readability counts.
- 8. Special cases aren't special enough to break the rules.
- 9. Although practicality beats purity.
- 10. Errors should never pass silently.
- 11. Unless explicitly silenced.
- 12. In the face of ambiguity, refuse the temptation to guess.
- 13. There should be one – and preferably only one – obvious way to do it.
- 14. Although that way may not be obvious at first unless you're Dutch.
- 15. Now is better than never.
- 16. Although never is often better than *right* now.
- 17. If the implementation is hard to explain, it's a bad idea.
- 18. If the implementation is easy to explain, it may be a good idea.
- 19. Namespaces are one honking great idea – let's do more of those!

35

Hello world!

```
print('Hello world')
```

A screenshot of a Python 3.7.1 Shell window. The window title is "Python 3.7.1 Shell". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area shows the following content:

```
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:57:15) [MSC v.1915 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print('hello world')
hello world
>>>
```

The status bar at the bottom right indicates "Ln: 5 Col: 4".

36

Hello world!

```
C++ "Hello World"

#include<iostream.h>
main()
{
    cout << "Hello World";
}
return 0
```

```
JAVA "Hello World"

class HelloWorldApp
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

```
PYTHON

print "Hello World"
```

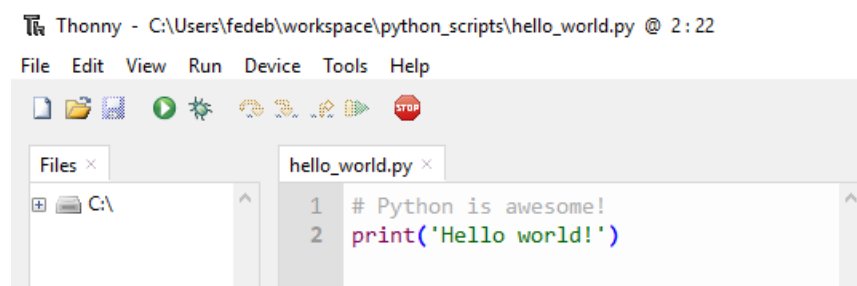


37

Comments

Remember to comment your code!

Comments are done with #



38

Printing to console

To show output from code to the user, use the print command

```
>>> print(3+2)
5
```

This is actually written to stdout

```
>>> 3+2
5
```

This is NOT written to stdout, just shown in the interpreter

You may also use **ipython**, which makes it easier to see the difference:

```
In [11]: 3+2
Out[11]: 5
```

“Out” means it’s an interaction within the shell only

```
In [12]: print(3+2)
5
```

No “Out”, it is actually shown to the user

Python programs

- a **program** is a sequence of definitions and commands
 - definitions **evaluated**
 - commands **executed** by Python interpreter in a shell
- **commands** (statements) instruct interpreter to do something
- can be typed directly in a **shell** or stored in a **file** that is read into the shell and evaluated

Objects

- a **program** manipulate **data objects**
- **objects** have a **type** that defines the kinds of things a program can do to them
- objects are
 - Scalar (cannot be subdivided)
 - Non-Scalar (have internal structure that can be accessed)

Expressions

combine objects and operators to form expressions

an expression has a value, which has a type

syntax for a simple expression

`<object> <operator> <object>`

First steps

First things first: **variables**.

- In software programming, variables are names used to hold one or more values.
- Instead of repeating these values in multiple places in your code, the variable holds such values.
- The variable is stored in computer memory once, and used wherever it is needed in your code.

43

First steps

- Python is **strongly typed** - types cannot change in unexpected ways
- Python is **dynamically typed** - runtime objects have a type, not variables
- Python is **implicitly typed** – variables are automatically converted from a data type to another
- Python is **case sensitive** - “a” and “A” are two different variables
- Python is **Object Oriented** – everything is an object

44

Types

Python has several built-in types

Numbers	1234, 3.1415, 3+4j, 0b111, Decimal(), Fraction()
Strings	'spam', "Bob's", b'a\x01c', u'sp\xc4m'
Lists	[1, [2, 'three'], 4.5], list(range(10))
Dictionaries	{'food': 'spam', 'taste': 'yum'}, dict(hours=10)
Tuples	(1, 'spam', 4, 'U'), tuple('spam'), namedtuple
Files	open('eggs.txt'), open(r'C:\ham.bin', 'wb')
Sets	set('abc'), {'a', 'b', 'c'}
Other core types	Booleans, types, None

45

Scalar Objects

- `int` – represent integers, ex. 5
- `float` – represent real numbers, ex. 3.27
- `bool` – represent Boolean values `True` and `False`
- `NoneType` – special and has one value, `None`
- can use `type()` to see the type of an object

```
>>> type(5)
int
>>> type(3.0)
float
```

What you write into the Python shell

What it answers back

46

Types and operators

Numbers

The most common types of numbers you will use are integer numbers *int* (e.g. 42) or floating point numbers *float* (e.g. 3.14).

You can perform usual operations on numbers such as $+$, $-$, $*$, $/$

```
>>> 40 + 2
42
>>> 2*3
6
>>> 4/3
1.3333333333333333
```

47

Types and operators

Numbers

`//` denotes the integer division

```
>>> 4/3
1.3333333333333333
>>> 4//3
1
```

48

Types and operators

Numbers

% denotes the modulo operation

```
>>> 21 % 7
0
>>> 53 % 5
3
```

49

Types and operators

Numbers

** denotes the power operation

```
>>> 2 ** 3
8
>>> 4 ** 2
16
>>> 3 ** 0
1
```

50

Types and operators

Numbers

Precedence can be given with parentheses

```
>>> 4 + 3 * 2
10
>>> 4 + (3 * 2)
10
>>> (4 + 3) * 2
14
```

51

Ints and floats' operators

- $i+j$ → the sum
 - $i-j$ → the difference
 - $i*j$ → the product
 - i/j → division
- If both are ints -> result is int**
If either or both are floats -> result is float
- result is always float**

52

Types and operators

Strings

Strings are sequences of alphanumeric characters delimited by ‘ ‘ or “ ”.

```
>>> 'egg'
'egg'
>>> "spam"
'spam'
```

53

Types and operators

Strings

You can concatenate strings using the + operator

```
>>> 'this lesson ' + 'is starting ' + "to be boring"
'this lesson is starting to be boring'
```

54

Types and operators

Strings

Strings can be replicated by multiplying them with a number

```
>>> 'spam'*5
'spamspamspamspamspam'
>>> 'spam'*5 + '!!!'
'spamspamspamspamspam!!!'
```

55

Types and operators

Strings

You can use `\` to escape special characters, like the single quote `'`

```
>>> 'it's a nice day
SyntaxError: invalid syntax
>>> 'it\' a nice day'
"it' a nice day"
```

56

Types and operators

Strings

You can also include quotes “ or single quotes ‘ in a string by using one to define the string

```
>>> "it's a nice day"  
"it's a nice day"  
>>> 'I "know" C'  
'I "know" C'
```

57

Types and operators

Booleans

Booleans can either be **True** or **False**

```
>>> True  
True  
>>> False  
False
```

58

Types and operators

Booleans

You can use the logical operators **or**, **and** and **not**

A	B	A AND B	A OR B	NOT A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

59

Types and operators

Booleans

You can use the logical operators **or**, **and** and **not**

```
>>> not True
False
>>> not False
True
```

60

Types and operators

Booleans

You can use the logical operators **or**, **and** and **not**

```
In [1]: exam_time = 15

In [2]: sleep_time = 8

In [3]: print(sleep_time > exam_time)
...:
False

In [1]: drive = True

In [2]: drink = False

In [3]: both = drink and drink

In [4]: print(both)
False
```

61

Types and operators

NoneType

Indicates “no value”, it is represented by the object **None**

```
>>> None
>>> type(None)
<class 'NoneType'>
```

62

Types and operators

type()

You can check the type of an object using the function **type()**

```
>>> type(None)
<class 'NoneType'>
>>> type(1)
<class 'int'>
>>> type(3.4)
<class 'float'>
>>> type('egg')
<class 'str'>
>>> type('True')
<class 'str'>
>>> type(True)
<class 'bool'>
```

63

Type conversions (cast)

We can convert object of one type to another

`float(3)` converts integer 3 to float 3.0

`int(3.9)` truncates float 3.9 to integer 3

64

Comparison operators

==

True if two variables are equal

!=

True if two variables are different

>

True if the first variable is bigger than the second

<

True if the first variable is smaller than the second

>=

True if the first variable is bigger or equal than the second

<=

True if the first variable is smaller or equal than the second

comparisons evaluate to a **Boolean**

65

Assignment

Using the operator = you can assign values to variables

```
>>> a = 1
>>> b = 2
>>> c = 3
>>> a + (b*c)
7
>>> s = 'spam'
>>> s = s * 2
>>> print(s)
spamspam
```

66

Assignment

Using the operator = you can assign values to variables

```
>>> a = 1
>>> b = a * 3
>>> a == b
False
>>> a == 1
True
```

67

Variables and values

equal sign is an assignment of a value to a variable name

Variable		Value
pi	=	3.14159

pi_approx = 22/7

Values are stored in computer memory

An assignment binds **name** to **value**

retrieve value associated with name or variable by invoking the name, by typing pi

68

Changing binding

We can re-bind variable names to different values using a new `=` statement

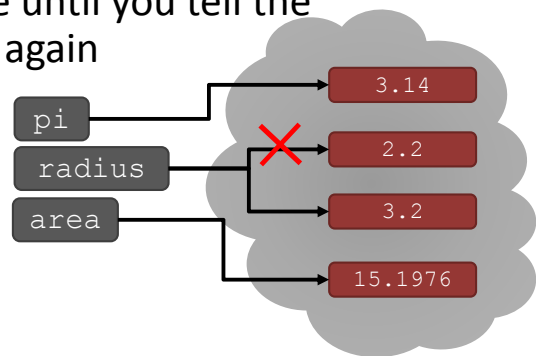
Previous value may still be stored in memory but lost the handle for it

Value for area does not change until you tell the computer to do the calculation again

```
pi = 3.14159
radius = 2.2

# area of circle
area = pi*(radius**2)
radius = radius+1
```

Value for area does not change until you tell the computer to do the calculation again



Sequences

Strings are actually sequences: a positionally ordered collection of objects (the characters).

Keep this in mind, since we will see different kinds of sequences.

Sequences support an amount of built-in operations and functions.

Sequences

Strings are actually sequences: a positionally ordered collection of objects (the characters).

Keep this in mind, since we will see different kinds of sequences.

Sequences support an amount of built-in operations and functions.

«A function is a group of related statements that perform a specific task»

A function is called using parentheses passing one or more arguments and returns a result.

We saw one already →

```
>>> print('hello world')
hello world
```

71

Sequences

The `len()` function tells us how long is a given sequence

```
>>> s = "rabbit"
>>> len(s)
6
```

This can only be used on sequences, like strings

```
>>> len(5)
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    len(5)
TypeError: object of type 'int' has no len()
```

72

Sequences

Sequences usually support also the *min()* and *max()* functions. For strings these return the minimum and maximum character in lexicographical order.

```
>>> s = "rabbit"
>>> len(s)
6
>>> min(s)
'a'
>>> max(s)
't'
```

This also works for numbers, but remember that each character is treated independently

```
>>> s = "99 67 3215"
>>> len(s)
10
```

73

Sequences

Sequences usually support also the *min()* and *max()* functions. For strings these return the minimum and maximum character in lexicographical order.

```
>>> s = "rabbit"
>>> len(s)
6
>>> min(s)
'a'
>>> max(s)
't'
```

This also works for numbers, but remember that each character is treated independently

```
>>> s = "99 67 3215"
>>> len(s)
10
>>> max(s)
'9'
```

74

Sequences

Sequences usually support also the *min()* and *max()* functions. For strings these return the minimum and maximum character in lexicographical order.

```
>>> s = "rabbit"
>>> len(s)
6
>>> min(s)
'a'
>>> max(s)
't'
```

This also works for numbers, but remember that each character is treated independently

```
>>> s = "99 67 3215"
>>> len(s)
10
>>> max(s)
'9'
>>> min(s)
' '
```

75

Indexing sequences

Sequences can be indexed using the indexing operator `[]`.

By passing a value (index) to the operator, it selects the correspondent value in the sequence.

```
>>> animal = 'rabbit'
>>> animal[0]
'r'
>>> animal[3]
'b'
>>> animal[10]
Traceback (most recent call last):
  File "<pyshell#60>", line 1, in <module>
    animal[10]
IndexError: string index out of range
```

76

Indexing sequences

Sequences can be indexed using the indexing operator [].

By passing a value (index) to the operator, it selects the correspondent value in the sequence.

Variables can be used to index sequences.

```
>>> animal = 'rabbit'
>>> index = 3
>>> animal[index]
'b'
```

77

Indexing sequences

Sequences can be indexed using the indexing operator [].

By passing a value (index) to the operator, it selects the correspondent value in the sequence.

Note that the last element of the string is in position *len(string)-1*

```
>>> animal = 'rabbit'
>>> string_len = len(animal)
>>> print(string_len)
6
>>> animal[string_len]
Traceback (most recent call last):
  File "<pyshell#79>", line 1, in <module>
    animal[string_len]
IndexError: string index out of range
>>> animal[string_len - 1]
't'
```

78

Indexing sequences

Sequences can be indexed using the indexing operator [].

By passing a value (index) to the operator, it selects the correspondent value in the sequence.

You can also index sequences from the bottom using negative indexes: -1 is the last element, -2 the second to last element, and so on.

```
>>> animal = 'rabbit'
>>> animal[-1]
't'
>>> animal[-2]
'i'
>>> animal[-3]
'b'
```

79

Slicing sequences

Indexing allows us to obtain an element of the sequence. The same operator [] can be used for *slicing*, i.e. taking a subsequence of the original sequence.

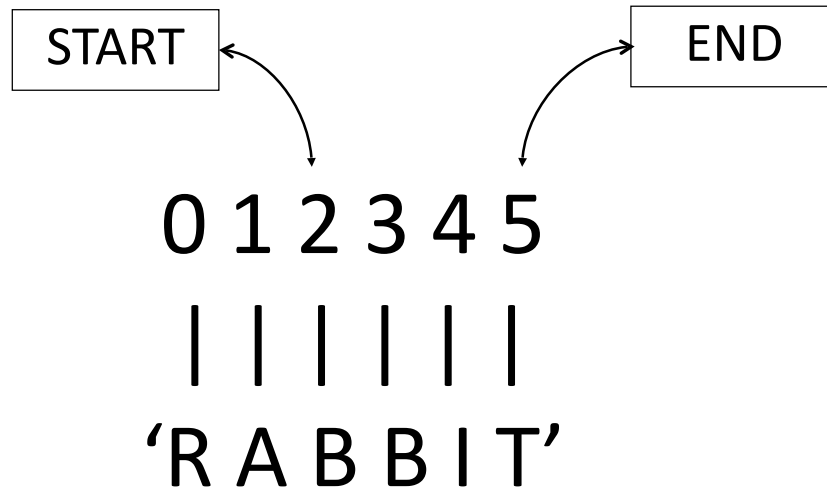
You need to pass the start and end indexes separated by the : operator, such as *string[start:end]*

This will create a new object with all the characters from *start* to *end-1*.

The element in position *start* is included, while the element in position *end* is not!

80

Slicing sequences



```
>>> animal = 'rabbit'
>>> animal[2:5]
'bbi'
```

81

Slicing sequences

You can avoid passing one of the *start* and *end* indexes to slice the sequence starting from the beginning or up till the end.

```
>>> animal = 'rabbit'
>>> animal[:4]
'rabb'
>>> animal[2:]
'bbit'
```

82

Formatting strings

Quite often you will need to format strings, inserting the values of other variables inside a string.

A better way to format a string is to use the *format()* function.

Placeholders can be inserted in a string using `{}` and then the variables can be passed as arguments to the *format()* function.

```
>>> name = 'Mike'
>>> age = 42
>>> print('{} is {} years old'.format(name, age))
Mike is 42 years old
```

87

Formatting strings

A better way to format a string is to use the *format()* function. Placeholders can be inserted in a string using `{}` and then the variables can be passed as arguments to the *format()* function.

You can also give explicit positional indexes to reuse variables.

```
>>> name = 'Mike'
>>> age = 42
>>> print('{0} is {1} years old. I said {1}!'.format(name, age))
Mike is 42 years old. I said 42!
```

88

Formatting strings

The good way to format strings, is to use the recently introduced f-strings (formatted string literals) from Python 3.6

```
# Python3 program introducing f-string
val = 'Geeks'
print(f"{val}for{val} is a portal for {val}.")

name = 'Om'
age = 22
print(f"Hello, My name is {name} and I'm {age} years old.")
```

Output

```
GeeksforGeeks is a portal for Geeks.
Hello, My name is Om and I'm 22 years old.
```

89

Formatting strings

F-string is typically faster and support formatting and expressions evaluation

```
english = 78
maths = 56
hindi = 85

print(f"Ram got total marks {english + maths + hindi} out of 300")
```

90

Formatting strings

F-string can be combined with any opening and closing quotation marks, as long as they are the same

```
print(f"'GeeksforGeeks'")  
print(f'""Geeks"for"Geeks""')  
print(f' ''Geeks'for'Geeks' ''')
```

Output

```
'GeeksforGeeks'  
Geeks"for"Geeks  
Geeks 'for'Geeks
```

91

Input strings

To input a string from the console use **input**

prints whatever is in the quotes
user types in something and hits enter

```
text = input("Type anything... ")  
print(5*text)
```

input gives you a string so must cast if working with numbers

```
num = int(input("Type a number... "))  
print(5*num)
```

92

Lists

Another type of sequence in Python is a **list**.

Lists, as the name suggests, are just sequences of elements that can be of different types.

You can define a list by writing its values separated by a comma and enclosed by square brackets.

```
>>> l = [1, 3, 5, 19]
>>> l
[1, 3, 5, 19]
```

93

Lists

```
>>> list1 = [1, 3, 5]

>>> print(list1)

[1, 3, 5]
>>> list2 = ['a', '45', 'qwe']

>>> print(list2)

['a', '45', 'qwe']
>>> list3 = [12, None, 5.4, [4, 8, 16], list2, 4]

>>> print(list3)

[12, None, 5.4, [4, 8, 16], ['a', '45', 'qwe'], 4]
```

94

Lists

Lists can be indexed and sliced just as other sequences.

```
>>> l = [1, 'apple', 32, 1.618, 'car']
>>> l[0]
1
>>> l[-1]
'car'
>>> l[3]
1.618
>>> l[:3]
[1, 'apple', 32]
```

95

Mutability vs. Immutability

Apart from the object types they can contain, Strings and Lists differ from a very important aspect: *mutability*.

String are immutable

Strings cannot be changed after they are created

Lists are mutable

Elements of a list can be changed after the list has been created

96

Immutability

```
>>> s = 'strings are immutable!'
>>> s[3] = 'v'
Traceback (most recent call last):
  File "<pyshell#189>", line 1, in <module>
    s[3] = 'v'
TypeError: 'str' object does not support item assignment
```

97

Mutability

```
>>> l = [12, 42, 56, 'asd', 'qwerty', 4.5]
>>> print(l)
[12, 42, 56, 'asd', 'qwerty', 4.5]
>>> l[2] = 'lists are mutable!'
>>> print(l)
[12, 42, 'lists are mutable!', 'asd', 'qwerty', 4.5]
```

98

References vs. Copies

Assignments always store references to objects, not copies of those objects.

Be aware that multiple assignments can generate multiple references to the same object.

This means that changing a mutable object in place may affect other references of the same object. This can be avoided asking Python explicitly to copy the object.

99

References vs. Copies

```
1 X = [1, 2, 3]
2 L = ['a', X, 'b']
3 D = [123, 5.4, X]
4
5 print('Original lists')
6 print('X: {}'.format(X))
7 print('L: {}'.format(L))
8 print('D: {}'.format(D))
9
10 X[1] = 999
11
12 print('\nAfter changing X')
13 print('X: {}'.format(X))
14 print('L: {}'.format(L))
15 print('D: {}'.format(D))
```

Shell ×

```
>>> %Run referencecopy.py
```

```
Original lists
X: [1, 2, 3]
L: ['a', [1, 2, 3], 'b']
D: [123, 5.4, [1, 2, 3]]

After changing X
X: [1, 999, 3]
L: ['a', [1, 999, 3], 'b']
D: [123, 5.4, [1, 999, 3]]
```

```
>>> |
```

You can use the **copy()** function to prevent this behaviour

```
1 X = [1, 2, 3]
2 L = ['a', X, 'b']
3 D = [123, 5.4, X.copy()]
4
5 print('Original lists')
6 print('X: {}'.format(X))
7 print('L: {}'.format(L))
8 print('D: {}'.format(D))
9
10 X[1] = 999
11
12 print('\nAfter changing X')
13 print('X: {}'.format(X))
14 print('L: {}'.format(L))
15 print('D: {}'.format(D))
```

Shell ×

```
>>> %Run referencecopy.py
```

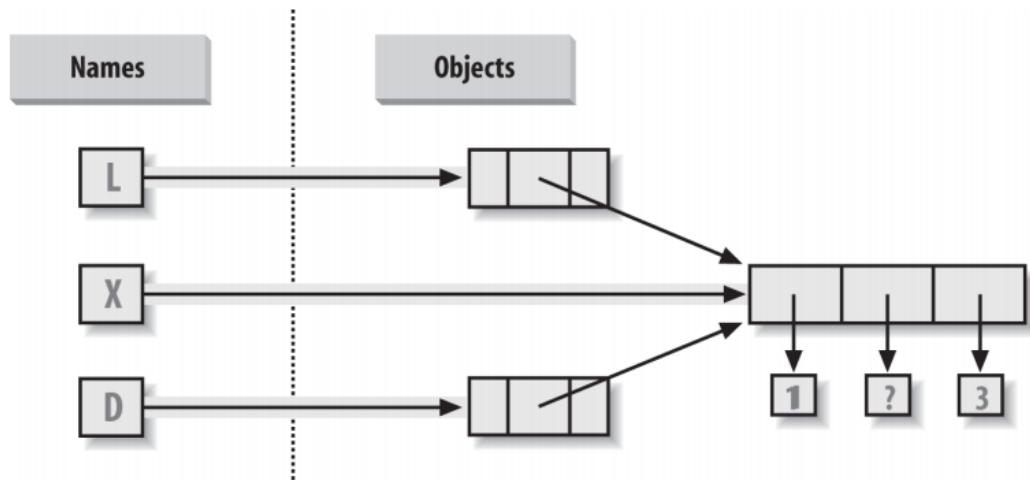
```
Original lists
X: [1, 2, 3]
L: ['a', [1, 2, 3], 'b']
D: [123, 5.4, [1, 2, 3]]

After changing X
X: [1, 999, 3]
L: ['a', [1, 999, 3], 'b']
D: [123, 5.4, [1, 2, 3]]
```

```
>>>
```

100

References vs. Copies



101

Operations on lists

Lists can be concatenated using the + operator.

```
>>> list1 = [1, 2, 3]
>>> list2 = [4, 5, 6]
>>> list3 = list1 + list2
>>> print(list3)
[1, 2, 3, 4, 5, 6]
```

102

Operations on lists

Elements can be added to a list using the **append** function

```
>>> list1 = [1, 2, 3]

>>> list1.append(4)

>>> print(list1)

[1, 2, 3, 4]
```

103

Operations on lists

list.append(obj) ↗
Appends object obj to list
list.count(obj) ↗
Returns count of how many times obj occurs in list
list.extend(seq) ↗
Appends the contents of seq to list
list.index(obj) ↗
Returns the lowest index in list that obj appears
list.insert(index, obj) ↗
Inserts object obj into list at offset index
list.pop(obj=list[-1]) ↗
Removes and returns last object or obj from list
list.remove(obj) ↗
Removes object obj from list
list.reverse() ↗
Reverses objects of list in place
list.sort([func]) ↗
Sorts objects of list, use compare func if given

104

CONTROL FLOW

105

Control flow

Up till now we have seen object types and some of the basic operations you can do with them.

This does not allow to develop even the most simple of algorithms.

An **algorithm** is a detailed series of instructions for carrying out an operation or solving a problem. All computer programs are based on algorithms.

An algorithm is like a recipe to solve a given problem.



106

Control flow

One of the most important elements to make a successful algorithm is to be able to control the flow of execution.

Control flow statements are mainly 3: **IF**, **FOR**, **WHILE**

107

Control flow - IF

The **IF** statement allows us to make a choice in the code.

The statement is written using the **if** keyword followed by a logical condition (it must return a boolean).

We can define a block of code to be executed only if the logical condition is True.

```
>>> a = 7

>>> b = 5

>>> if a > b:
    print('a is bigger')

a is bigger
```

108

Control flow - IF

The if statement ends with : and the code block to be executed is delimited by a different **indentation**.

All code blocks in Python are defined by indentation. This helps keeping the code clean and understandable.

```
>>> a = 7
>>> b = 5
>>> if a > b:
    print('a is bigger')
a is bigger
```



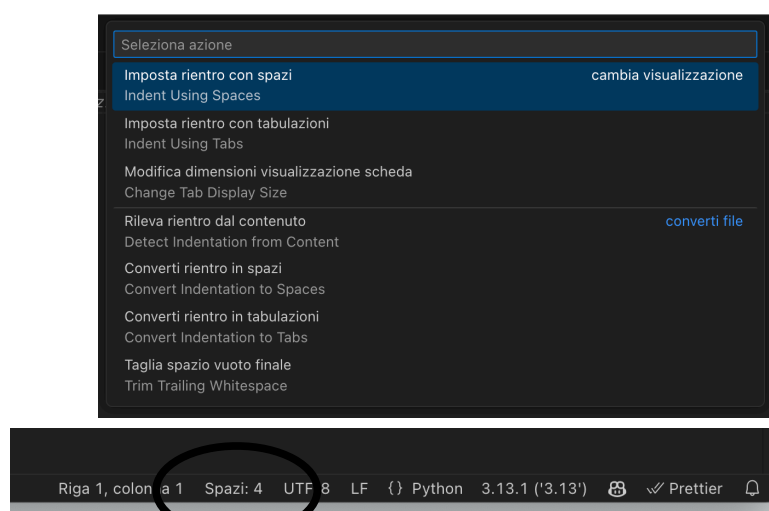
109

Indentation

Indentation matters in Python!

It may be in a fixed amount of spaces (4 typical or 8) or by using tabulation (that can be “converted” to spaces arbitrarily by the IDE).

They cannot be mixed!



110

Control flow – IF... ELSE

If the condition of the **IF** clause is not satisfied we can tell Python to execute another block of code using the keyword **else**

```
1 a = 3
2 b = 5
3
4 if a > b:
5     print('a is bigger')
6 else:
7     print('a is smaller')
```

Shell ×

Python 3.7.2 (bundled)

```
>>> %Run if_else.py
a is smaller
```

```
1 a = 7
2 b = 5
3
4 if a > b:
5     print('a is bigger')
6 else:
7     print('a is smaller')
```

Shell ×

```
>>> %Run if_else.py
a is bigger
```

111

Control flow – IF... ELIF... ELSE

You can add multiple mutually exclusive conditions using **if ... elif ... else**

```
1 a = 7
2 b = 5
3
4 if a > b:
5     print('a is bigger')
6 elif a < b:
7     print('a is smaller')
8 else:
9     print('a and b are the same')
```

Shell ×

```
>>> %Run elif.py
a is bigger
```

```
1 a = 3
2 b = 15
3
4 if a > b:
5     print('a is bigger')
6 elif a < b:
7     print('a is smaller')
8 else:
9     print('a and b are the same')
```

Shell ×

```
>>> %Run elif.py
a is smaller
```

```
1 a = 4
2 b = 4
3
4 if a > b:
5     print('a is bigger')
6 elif a < b:
7     print('a is smaller')
8 else:
9     print('a and b are the same')
```

Shell ×

```
>>> %Run elif.py
a and b are the same
```

112

= VS ==

```
x = float(input("Enter a number for x: "))
y = float(input("Enter a number for y: "))
```

```
if x == y: ← What if == is changed to = ?
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
else:
    print("y is smaller")

print("thanks!")
```

= VS ==

```
x = float(input("Enter a number for x: "))
y = float(input("Enter a number for y: "))
```

```
if x == y: ← What if == is changed to = ?
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
else:
    print("y is smaller")

print("thanks!")
```

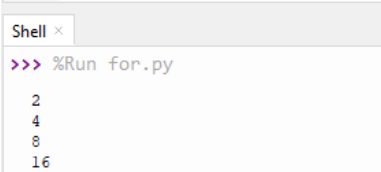
-> SyntaxError

Control flow - FOR

Very often you will need to loop over a sequence (let it be a string, a list, etc).

You can loop over a sequence using the **for ... in** statement.

```
1 my_list = [2, 4, 8, 16]
2
3 for i in my_list:
4     print(i)
```



Using the **for** statement you will access all elements of the sequence one by one. In this example, at each iteration the variable **i** will be replaced with the next element in the sequence.

The range() function

The **range()** function is often used in combination with for loops.

It takes up to 3 arguments:

- 1 argument: **range(end)**
- 2 arguments: **range(start, end)**
- 3 arguments: **range(start, end, step)**

range() generates a list of numbers between **start** (included) and **end** (not included). If only one argument is passed to the function, it returns the numbers between **0** and **end-1**.

The range() function

```
1 for i in range(5):  
2     print(i)
```

Shell ×

```
>>> %Run range.py  
0  
1  
2  
3  
4
```

```
1 for i in range(3,8):  
2     print(i)
```

Shell ×

```
>>> %Run range.py  
3  
4  
5  
6  
7
```

```
1 for i in range(4, 10, 2):  
2     print(i)
```

Shell ×

```
>>> %Run range.py  
4  
6  
8
```

117

Strings and loops

These two code snippets do the same thing but the bottom one is more “pythonic”

```
s = "abcdefgh"

for index in range(len(s)):
    if s[index] == 'i' or s[index] == 'u':
        print("There is an i or u")

for char in s:
    if char == 'i' or char == 'u':
        print("There is an i or u")
```

118

Iterables

Everything that can be used with a for loop is called an *iterable*.

Iterables are not always indexable, they might not have a length and they even might not be finite.

An iterable object can be passed to the built-in function **iter()** to get an **iterator** to loop over it.

An iterator has the job of returning the next item of the iterable. These elements can be obtained in sequence using another built-in function called **next()**.

119

Iterables

```
>>> r = range(4)

>>> iterator = iter(r)

>>> next(iterator)

0
>>> next(iterator)

1
>>> next(iterator)

2
>>> next(iterator)

3
>>> next(iterator)

Traceback (most recent call last):
  File "<pyshell#296>", line 1, in <module>
    next(iterator)
StopIteration
```

120

The enumerate() function

The *enumerate()* function wraps an iterable object in order to return also an index

```
1 for r in range(10):
2     print(r)
```

Shell ×

```
>>> %Run enumerate.py
0
1
2
3
4
5
6
7
8
9
```

```
1 for i, r in enumerate(range(3, 10)):
2     print('element in position {}: {}'.format(i,r))
```

Shell ×

```
>>> %Run enumerate.py
element in position 0: 3
element in position 1: 4
element in position 2: 5
element in position 3: 6
element in position 4: 7
element in position 5: 8
element in position 6: 9
```

121

Control flow - WHILE

With the **while()** loop we can execute a set of statements as long as a condition is true.

```
1 n = 10
2 i = 0
3 while i < 10:
4     print(i)
5     i += 1
```

Shell ×

```
>>> %Run while.py
0
1
2
3
4
5
6
7
8
9
```

122

Control flow - break

Inside *while* and *for* loops you might want to stop the execution and exit from the loop if some condition is verified.

You can check the condition as usual with an *if* statement, and you can exit the loop with the **break** statement.

```
1 n = 10
2 i = 0
3 while i < 10:
4     if i == 5:
5         break
6     print(i)
7     i += 1
```

Shell ×

```
>>> %Run while.py
0
1
2
3
4
>>>
```

123

Control flow - continue

On the other hand, you might want to skip just an iteration. Inserting a **continue** statement in a loop will cause Python to skip the current iteration and move on to the next one.

```
1 n = 10
2 i = 0
3 for i in range(10):
4     if i % 2 == 0:
5         continue
6     print(i)
7
```

Shell ×

```
>>> %Run continue.py
1
3
5
7
9
>>>
```

124

List Comprehensions

List comprehensions are a way to build a new list by running an expression on each item in a sequence, one at a time.

A list comprehension can always be written as an explicit for loop, but...

- Less code
- Easier to read
- Possibly faster
- More *pythonic*

125

List Comprehensions

Suppose to define a matrix m , built from a set of rows $r1$, $r2$, $r3$.

```
>>> r1 = [1, 3, 5, 9]
>>> r2 = [12, 18, 11, 10]
>>> r3 = [25, 27, 22, 21]
>>> m = [r1, r2, r3]
>>> print(m)
[[1, 3, 5, 9], [12, 18, 11, 10], [25, 27, 22, 21]]
>>>
```

126

List Comprehensions

Keep in mind that this is not how you would normally represent a matrix. Scientific libraries such as *numpy* can be used for represent matrices... We will see this later on...

```
>>> r1 = [1, 3, 5, 9]
>>> r2 = [12, 18, 11, 10]
>>> r3 = [25, 27, 22, 21]
>>> m = [r1, r2, r3]
>>> print(m)
[[1, 3, 5, 9], [12, 18, 11, 10], [25, 27, 22, 21]]
>>>
```

127

List Comprehensions

What if you wanted to create a new list containing only the i-th column?

```
>>> r1 = [1, 3, 5, 9]
>>> r2 = [12, 18, 11, 10]
>>> r3 = [25, 27, 22, 21]
>>> m = [r1, r2, r3]
>>> print(m)
[[1, 3, 5, 9], [12, 18, 11, 10], [25, 27, 22, 21]]
>>>
```

128

List Comprehensions

What if you wanted to create a new list containing only the i-th column?

```
1 r1 = [1, 3, 5, 9]
2 r2 = [12, 18, 11, 10]
3 r3 = [25, 27, 22, 21]
4 m = [r1, r2, r3] # m is a 3x4 matrix (3 rows, 4 columns)
5
6 i = 0 # select the first column
7
8 i_column = []
9 for r in range(len(m)):
10     i_column.append(m[r][0])
11
12 print(i_column)
```

Shell x

```
>>> %Run matrix_column.py
```

```
[1, 12, 25]
```

```
>>>
```

129

List Comprehensions

There is a much easier and compact way to write this, using **list comprehensions**.

```
1 r1 = [1, 3, 5, 9]
2 r2 = [12, 18, 11, 10]
3 r3 = [25, 27, 22, 21]
4 m = [r1, r2, r3] # m is a 3x4 matrix (3 rows, 4 columns)
5
6 i = 0 # select the first column
7
8 i_column = [r[0] for r in m]
9
10 print(i_column)
11
```

Shell x

```
>>> %Run matrix_column_comprehension.py
```

```
[1, 12, 25]
```

```
>>>
```

130

List Comprehensions

A list comprehension is a tool that allows us to create a new list in a concise way. Comprehensions are also useful to filter and/or transform elements.

Output Expression	Variable	Reference Sequence	Predicate (Optional)
↓	↓	↓	↓
output = [(number-1) for number in reference if number % 2 == 0]			

131

List Comprehensions

Example: split a string in a list of characters

```
>>> string = 'python'

>>> print(string)

python
>>> chars = [c for c in string]

>>> print(chars)

['p', 'y', 't', 'h', 'o', 'n']
```

132

List Comprehensions

Example: split a string in the list of characters in even positions

```
>>> string = 'python'

>>> print(string)

python
>>> chars = [c for c in string]

>>> print(chars)

['p', 'y', 't', 'h', 'o', 'n']
>>>

>>> even_chars = [c for i, c in enumerate(string) if i%2 == 0]

>>> print(even_chars)

['p', 't', 'o']
```

3

Some Exercises

- 1) Given a list of numbers from 1 to 10, generate a new list with the same numbers in inverted order using loops. [1,2,3] → [3,2,1]
- 2) Given a list of numbers from 1 to 10, create a new list of odd numbers only using comprehensions.
- 3) Given a list of numbers a = [1,9,3,6,2,5,8,7,1,8,2] generate a new list containing the indices i such that the square of the i-th number is greater than the sum of the values from 0 to i using comprehensions.
 - Hint: you can use the sum() built-in function for summing. Check in the documentation
- 4) Given a list of numbers A = [2,1,4,2,3,4,5,6,7,8,9,10] generate a new list B where the i-th element of B is the sum of the previous **even** elements in A i.e. from 0 to i, using loops.
- 5) Given the string s = 'Python is not that hard', generate a new string (string not list!!) containing only the consonants. Use comprehensions
 - Hint: convert the list to a string using the join() built-in function